# BUILDING AND SECURING A REST API

MoMo SMS Data Processing System

Team: BroCode

# 1. INTRODUCTION TO API SECURITY

Application Programming Interfaces (APIs) are the backbone of modern software systems. They allow different applications to communicate with each other, share data, and trigger actions across services. As APIs handle increasingly sensitive data, securing them has become one of the most critical responsibilities in software development.

In this project, we built a REST API that exposes mobile money transaction data parsed from SMS records. Because this data contains financial information including sender names, amounts, and account balances, securing access to it is not optional. An unsecured API would allow anyone with network access to read, modify, or delete transaction records.

API security operates on several layers. Authentication establishes who is making a request. Authorization determines what that user is allowed to do. Transport security ensures that data cannot be intercepted in transit. Input validation protects the server from malformed or malicious payloads.

For this implementation, we applied Basic Authentication across all endpoints. Every request must include a valid username and password encoded in the Authorization header. Requests that omit credentials or provide incorrect ones are rejected with a 401 Unauthorized response before any data is accessed or modified. We also implemented input validation on POST and PUT requests to reject malformed JSON bodies with a 400 Bad Request response.

While Basic Authentication provides a working layer of access control, it has well-documented limitations that are addressed in Section 4 of this report.

# 2. API ENDPOINT DOCUMENTATION

Base URL: http://localhost:8000
Auth: Basic Authentication
Format: All responses are JSON

## GET /transactions

Returns a list of all transaction records in the dataset.

**Request:**

Method: GET
Endpoint: /transactions
Headers: Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=

**Response (200 OK):**


```
[
  {
    "id": 1,
    "address": "M-Money",
    "readable_date": "10 May 2024 4:30:58 PM",
    "transaction_type": "incoming",
    "amount": 2000,
    "currency": "RWF",
    "sender": "Jane Smith",
    "receiver": null,
    "counterpart_phone": "*********013",
    "transaction_date": "2024-05-10 16:30:51",
    "new_balance": 2000,
```

```
    "transaction_id": "76662021700"
  },
  ...
]
```


**Error Codes:**

401 Unauthorized - Missing or invalid credentials

## GET /transactions/{id}

Returns a single transaction matching the given ID.

**Request:**

Method: GET

Endpoint: /transactions/1

Headers: Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=

**Response (200 OK):**


```
{
  "id": 1,
  "address": "M-Money",
  "readable_date": "10 May 2024 4:30:58 PM",
  "transaction_type": "incoming",
  "amount": 2000,
  "currency": "RWF",
  "sender": "Jane Smith",
  "receiver": null,
  "counterpart_phone": "*********013",
  "transaction_date": "2024-05-10 16:30:51",
  "new_balance": 2000,
  "transaction_id": "76662021700"
}
```


**Error Codes:**

400 Bad Request - ID is not a number

401 Unauthorized - Missing or invalid credentials

404 Not Found - No transaction with that ID exists

## POST /transactions

Adds a new transaction record to the dataset.

**Request:**

Method: POST

Endpoint: /transactions

Headers: Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=

Content-Type: application/json

Body:

```
{
  "amount": 5000,
  "transaction_type": "incoming",
  "sender": "John Doe",
  "receiver": "Me",
  "currency": "RWF"
}
```

Required fields: amount, transaction_type, sender, receiver

**Response (201 Created):**

```
{
  "amount": 5000,
  "transaction_type": "incoming",
  "sender": "John Doe",
  "receiver": "Me",
  "currency": "RWF",
  "id": 1692
}
```

**Error Codes:**

400 Bad Request - Invalid JSON or missing required fields

401 Unauthorized - Missing or invalid credentials

## PUT /transactions/{id}

Updates one or more fields on an existing transaction.

**Request:**

Method: PUT

Endpoint: /transactions/1

Headers: Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=

Content-Type: application/json

Body:

```
{
  "amount": 9999,
  "sender": "Updated Sender"
}
```

**Response (200 OK):**

```
{
  "id": 1,
  "transaction_type": "incoming",
  "amount": 9999,
  "sender": "Updated Sender",
  ...
}
```

**Error Codes:**

400 Bad Request - ID is not a number or invalid JSON body

401 Unauthorized - Missing or invalid credentials

404 Not Found - No transaction with that ID exists

## DELETE /transactions/{id}

Removes a transaction from the dataset and returns the deleted record as confirmation.

**Request:**

Method: DELETE

Endpoint: /transactions/1

Headers: Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=

**Response (200 OK):**

```
{
  "message": "Transaction 1 deleted.",
  "deleted": {
    "id": 1,
    "transaction_type": "incoming",
    "amount": 2000,
    "sender": "Jane Smith",
    ...
  }
}
```

**Error Codes:**

400 Bad Request - ID is not a number

401 Unauthorized - Missing or invalid credentials

404 Not Found - No transaction with that ID exists

**Error Code Summary**

200 OK Successful GET, PUT, or DELETE

| | | |
|---|---|---|
| 201 | Created | Successful POST |
| 400 | Bad Request | Non-numeric ID or malformed JSON |
| 401 | Unauthorized | Incorrect or missing credentials |
| 404 | Not Found | Transaction or endpoint does not exist |

# 3. DSA COMPARISON: LINEAR SEARCH VS DICTIONARY LOOKUP

## Overview

To demonstrate efficient data retrieval, we implemented and compared two search approaches on our transaction dataset:linear search over a list, and dictionary lookup using a hash map. Both were tested against the same set of 20 target IDs spread across early, middle, and late positions in the dataset to ensure a fair comparison.

Each timing was averaged over 1000 runs using Python's time.perf_counter() to reduce variance from system noise.

## Results Table:

| ID | Linear (ms) | Dict (ms) | Faster by |
|---|---|---|---|
| 1 | 0.000079 | 0.000056 | 1.4x |
| 50 | 0.001160 | 0.000096 | 12.1x |
| 100 | 0.002180 | 0.000055 | 40.0x |
| 150 | 0.002993 | 0.000050 | 60.0x |
| 200 | 0.004571 | 0.000054 | 84.3x |
| 250 | 0.005504 | 0.000059 | 93.3x |
| 300 | 0.006524 | 0.000057 | 113.9x |
| 350 | 0.007990 | 0.000054 | 147.7x |
| 400 | 0.008577 | 0.000052 | 164.0x |
| 450 | 0.009950 | 0.000057 | 175.8x |
| 500 | 0.010481 | 0.000054 | 194.1x |
| 600 | 0.012900 | 0.000055 | 233.7x |
| 700 | 0.015997 | 0.000095 | 167.5x |
| 800 | 0.017304 | 0.000055 | 312.3x |
| 900 | 0.018575 | 0.000054 | 346.5x |
| 1000 | 0.022202 | 0.000056 | 393.6x |
| 1100 | 0.023912 | 0.000056 | 430.1x |
| 1200 | 0.026836 | 0.000055 | 491.5x |
| 1400 | 0.031007 | 0.000058 | 530.9x |
| 1691 | 0.037271 | 0.000056 | 667.9x |

## Analysis

The results confirm what algorithmic theory predicts. Linear search performance degrades as the target ID increases because the function must scan through more records before finding a match. Searching for record 1 requires one comparison while searching for record 1691 may require up to 1691 comparisons in the worst case. This gives linear search a time complexity of O(n), where n is the number of records in the dataset.

Dictionary lookup, on the other hand, takes the same amount of time regardless of which record is being retrieved. Python dictionaries are implemented as hash tables. When a key is provided, Python computes its hash and uses that to jump directly to the memory location of the value, bypassing any scanning. This gives dictionary lookup a time complexity ofO(1), meaning constant time.

The practical implication is significant at scale. With 1691records our dataset is relatively small. In a production system handling millions of transactions, linear search would become unusably slow while dictionary lookup would remain fast.

## Alternative Data Structure

A Binary Search Tree (BST) would be a strong alternative for this use case. Unlike a dictionary, a BST maintains records in sorted order, which makes it efficient not just for single lookups but also for range queries such as finding all transactions between two dates. A balanced BST such as anAVL tree or Red-Black tree achieves O(log n) lookup time,which is slower than dictionary lookup for single records but far more flexible for queries that involve ordering oranges. For a financial dataset where users frequently filter by date or amount, a BST would offer better overall utility than a plain dictionary.

# 4. BASIC AUTH LIMITATIONS AND STRONGER ALTERNATIVES

## How Basic Authentication Works

Basic Authentication works by encoding the username and password in Base64 and attaching them to every HTTP request in the Authorization header. The server decodes the header,extracts the credentials, and either grants or denies access.

For example, the credentials admin:password123 become:

```
Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
```

This is the approach we implemented in our API.

## Why Basic Auth Is Weak

Basic Authentication has several serious weaknesses that make it unsuitable for production systems. First, Base64 is not encryption. It is simply an encoding scheme that any developer can reverse in seconds. If an attacker intercepts a request, they immediately have the plaintext username and password.

This makes Basic Auth completely dependent on HTTPS to be safe at all, and even then credentials are exposed in every single request.

Second, credentials cannot be selectively revoked. If a client's credentials are compromised, the only option is to change the password for everyone using that account. There is no concept of sessions, tokens, or expiry.

Third, there is no concept of scope. Basic Auth grants full access or no access. A read-only client and an admin client must share the same credential system with no way to differentiate permissions at the protocol level.

Fourth, every request carries the full credentials. This increases the attack surface with every API call made.

## Stronger Alternative: JWT (JSON Web Tokens)

JWT authentication works by having the client log in once with their credentials. The server verifies those credentials and issues a signed token that expires after a set time period, such as one hour. The client then attaches this token to subsequent requests instead of their raw credentials.

The advantages are significant. Tokens expire automatically,limiting the damage if one is stolen. The server does not need to look up the user in a database on every request sinceThe token itself carries verified claims. Tokens can also carry scope information, allowing the server to grant different levels of access to different clients from the authentication system.